

Experiment Name: Shortest-Job-First (Preemptive and Non-Preemptive) Scheduling

Problem 1: Shortest-Job-First (Non-Preemptive Algorithm)

Description

This experiment demonstrates the Shortest-Job-First (SJF) Non-Preemptive scheduling algorithm.

In this algorithm, the CPU always selects the job with the smallest burst time among all processes that have already arrived. Once a process starts executing, it continues until completion without interruption.

The goal of this experiment is to observe how SJF minimizes waiting and turnaround times by prioritizing shorter tasks.

Source Code

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
struct P
```

```
{  
    int id;  
    int at;  
    int bt;  
    int rt;  
    int ct;  
    int wt;  
    int tat;  
};
```

```
int32_t main()
```

```
{  
    int n;  
    if (!(cin >> n))  
        return 0;  
    vector<P> p(n);  
  
    for (int i = 0; i < n; i++)  
    {
```

```
p[i].id = i;
cin >> p[i].at >> p[i].bt;
p[i].rt = p[i].bt;
p[i].ct = 0;
p[i].wt = 0;
p[i].tat = 0;
}
```

```
sort(p.begin(), p.end(), [](const P &a, const P &b)
{
    if(a.at == b.at) return a.id < b.id;
    return a.at < b.at; });
```

```
set<tuple<int, int, int>> rd;
int t = p[0].at;
int i = 0;
int dn = 0;
```

```
while (dn < n)
{
    while (i < n && p[i].at <= t)
    {
        rd.insert( {p[i].bt, p[i].at, i} );
        i++;
    }
}
```

```
if (rd.empty())
{
    t++;
    continue;
}
```

```
}
```

```
auto it = rd.begin();
```

```
auto [burst, arr, idx] = *it;
```

```
rd.erase(it);
```

```
t += p[idx].rt;
```

```
p[idx].rt = 0;
```

```
p[idx].ct = t;
```

```
p[idx].wt = p[idx].ct - p[idx].at - p[idx].bt;
```

```
p[idx].tat = p[idx].ct - p[idx].at;
```

```
dn++;
```

```
while (i < n && p[i].at <= t)
```

```
{
```

```
    rd.insert( {p[i].bt, p[i].at, i} );
```

```
    i++;
```

```
}
```

```
}
```

```
vector<P> res = p;
```

```
sort(res.begin(), res.end(), [](const P &a, const P &b)
```

```
    { return a.id < b.id; });
```

```
double total_wt = 0;
```

```
double total_tat = 0;
```

```
cout << "Process\tWaiting\tTurnaround\n";
```

```
for (auto &pr : res)
```

```
{
```

```

    cout << pr.id + 1 << "\t" << pr.wt << "\t" << pr.tat << "\n";

    total_wt += pr.wt;
    total_tat += pr.tat;
}

cout << "Average waiting time = " << (total_wt / n) << "\n";
cout << "Average turnaround time = " << (total_tat / n) << "\n";

return 0;
}

```

Input

```

4
0 8
1 4
2 9
4 5

```

Output

Process	Waiting	Turnaround
1	0	8
2	7	11
3	15	24
4	8	13

Average waiting time = 7.5

Average turnaround time = 14

Conclusion

This experiment illustrates the implementation of the non-preemptive Shortest-Job-First scheduling method using C++. Each process is selected based on the smallest burst time among the available processes, and once execution begins it cannot be interrupted. The program calculates waiting time, turnaround time, and the averages for both metrics. The results show that shorter tasks significantly reduce the average waiting time, demonstrating why SJF is an optimal scheduling approach for minimizing delays when preemption is not required.

Problem 2: Shortest-Job-First (Preemptive Algorithm)

Description

This experiment demonstrates the Shortest-Remaining-Time-First (SRTF) scheduling method, which is the preemptive version of Shortest-Job-First. In this algorithm, the CPU always selects the process with the lowest remaining time. If a new process arrives with a shorter remaining time than the currently running one, an immediate preemption occurs.

This experiment highlights how preemption further reduces the waiting time of newly arriving short processes and illustrates how remaining time is updated at each time unit.

Source Code

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
struct P
```

```
{
```

```
    int id;
```

```
    int at;
```

```
    int bt;
```

```
    int rt;
```

```
    int ct;
```

```
    int wt;
```

```
    int tat;
```

```
};
```

```
int32_t main()
```

```
{
```

```
    int n;
```

```
    if (!(cin >> n))
```

```
        return 0;
```

```
    vector<P> p(n);
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
p[i].id = i;
cin >> p[i].at >> p[i].bt;
p[i].rt = p[i].bt;
p[i].ct = 0;
p[i].wt = 0;
p[i].tat = 0;
}
```

```
sort(p.begin(), p.end(), [](const P &a, const P &b)
{
    if (a.at == b.at) return a.id < b.id;
    return a.at < b.at; });
```

```
set<tuple<int, int, int>> rd;
int t = p[0].at;
int i = 0;
int dn = 0;
```

```
while (dn < n)
{
    while (i < n && p[i].at <= t)
    {
        rd.insert({p[i].rt, p[i].at, i});
        i++;
    }
}
```

```
if (rd.empty())
{
    t++;
    continue;
}
```

```
}
```

```
auto it = rd.begin();
```

```
auto [rem, arr, idx] = *it;
```

```
rd.erase(it);
```

```
p[idx].rt--;
```

```
t++;
```

```
while (i < n && p[i].at <= t)
```

```
{
```

```
    rd.insert({p[i].rt, p[i].at, i});
```

```
    i++;
```

```
}
```

```
if (p[idx].rt == 0)
```

```
{
```

```
    p[idx].ct = t;
```

```
    p[idx].wt = p[idx].ct - p[idx].at - p[idx].bt;
```

```
    p[idx].tat = p[idx].ct - p[idx].at;
```

```
    dn++;
```

```
}
```

```
else
```

```
{
```

```
    rd.insert({p[idx].rt, p[idx].at, idx});
```

```
}
```

```
}
```

```
vector<P> res = p;
```

```
sort(res.begin(), res.end(), [](const P &a, const P &b)
```

```

        { return a.id < b.id; });

double total_wt = 0;
double total_tat = 0;

cout << "Process\tWaiting\tTurnaround\n";
for (auto &pr : res)
{
    cout << pr.id + 1 << "\t" << pr.wt << "\t" << pr.tat << "\n";
    total_wt += pr.wt;
    total_tat += pr.tat;
}

cout << "Average waiting time = " << (total_wt / n) << "\n";
cout << "Average turnaround time = " << (total_tat / n) << "\n";

return 0;
}

```

Input

```

4
0 8
1 4
2 9
3 5

```

Output

Process	Waiting	Turnaround
1 9	17	
2 0	4	
3 15	24	
4 2	7	

Average waiting time = 6.5

Average turnaround time = 13

Conclusion

The preemptive SJF algorithm, also known as SRTF, selects processes based on the smallest remaining burst time and allows interruption whenever a shorter process arrives. The results show that preemption can significantly reduce the waiting time of shorter incoming tasks compared to the non-preemptive version. This experiment clearly demonstrates how remaining times are updated, how processes re-enter the ready queue, and how preemption helps achieve lower average waiting times in dynamic arrival scenarios.